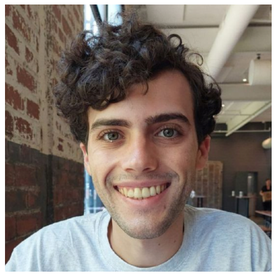
ORQ:
Complex Analytics on Private Data with Strong Security Guarantees

SOSP 2025

Eli Baum
Boston University



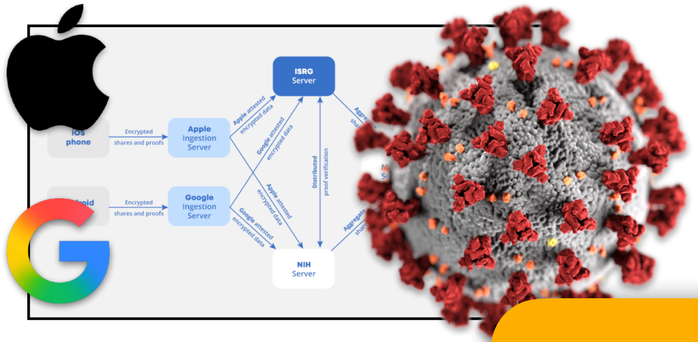



joint work with Sam Buxbaum, Nitin Mathai, Muhammad Faisal, Vasia Kalavri, Mayank Varia, and John Liagouris

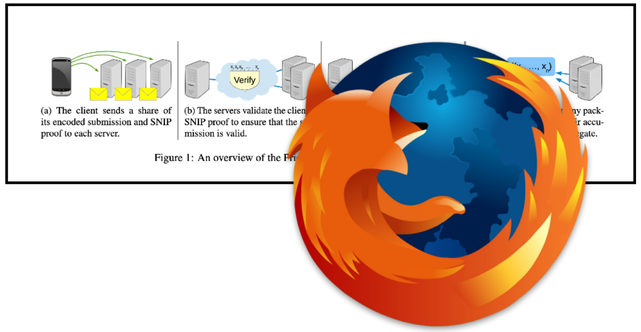
New England Database Day — 16 Jan 2026

Motivation: Collaborative Analytics on Sensitive Data

Common interest



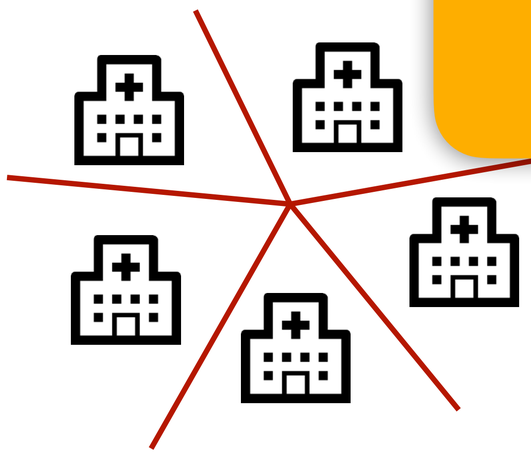
Disease Surveilla



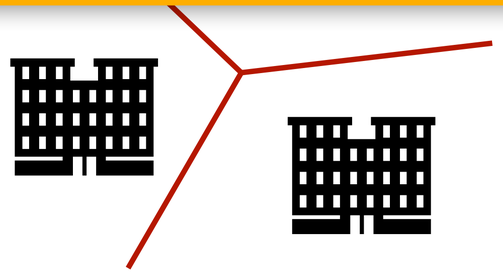
Browser Telemetry

Collaborative analytics requires securely joining data from multiple sources!

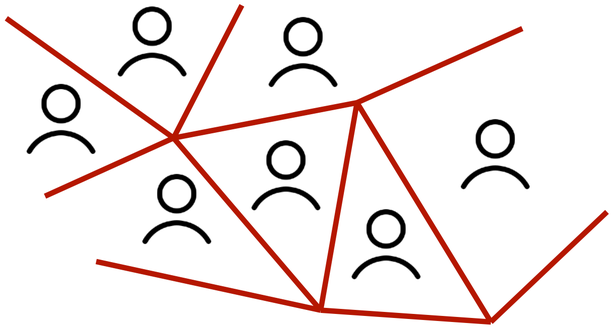
Private data



Hospitals



Employers



End Users

Outline

- Background on Oblivious Computation
- Our Approach
- The ORQ Framework
- Results

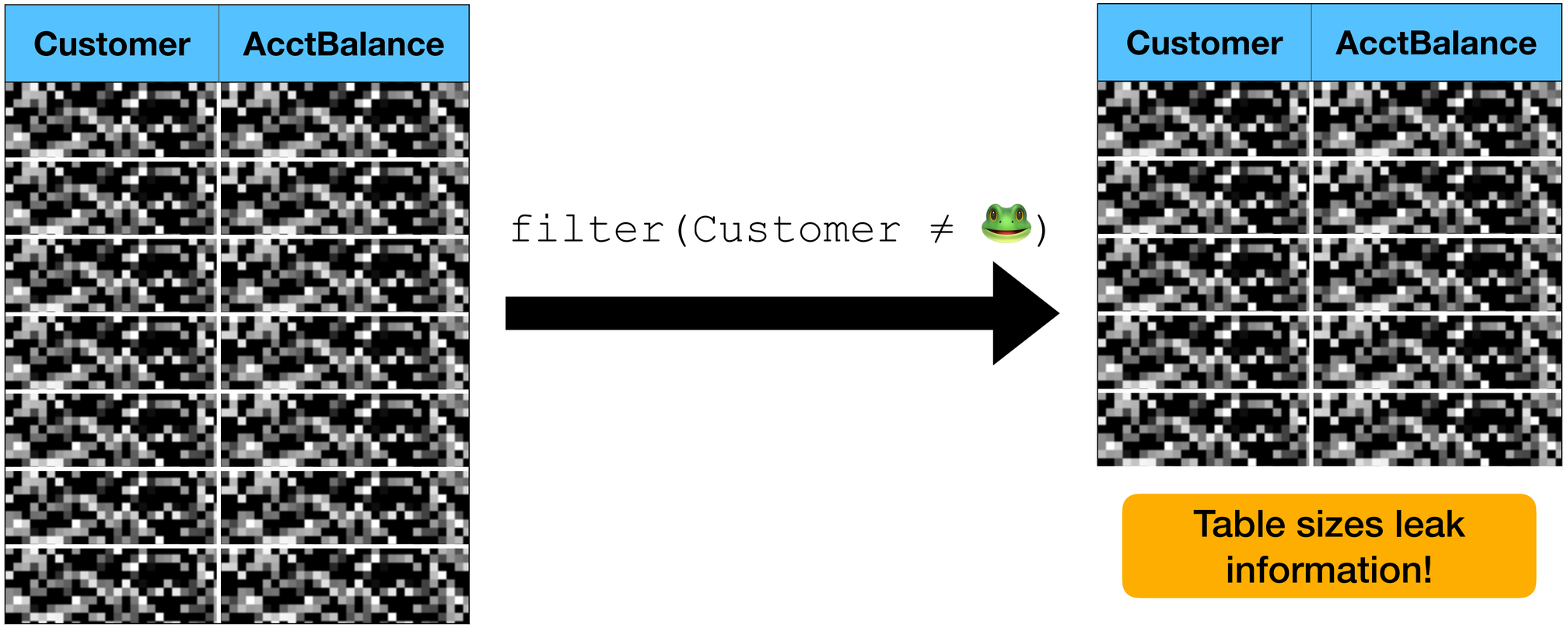
Plaintext Analytics

Customer	AcctBalance
🤡	$1000
😈	$2000
🤡	$500
🐸	$60
🥶	$8000
🥶	$700
🐸	-$200

Customer	AcctBalance
🤡	$1000
😈	$2000
🤡	$500
🥶	$8000
🥶	$700

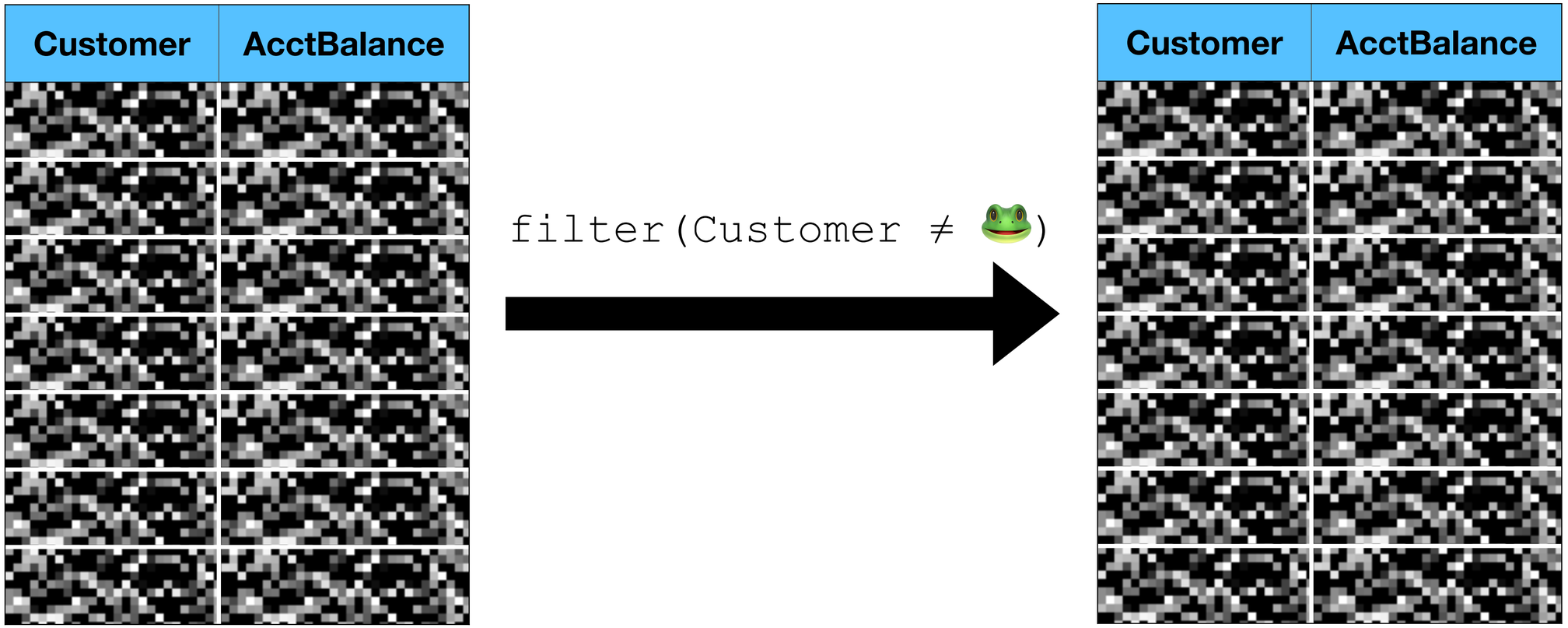
Plaintext Analytics

Adversary View



Oblivious Analytics

Complex oblivious operators must preserve **worst-case sizes**



Oblivious Joins with Cartesian Product

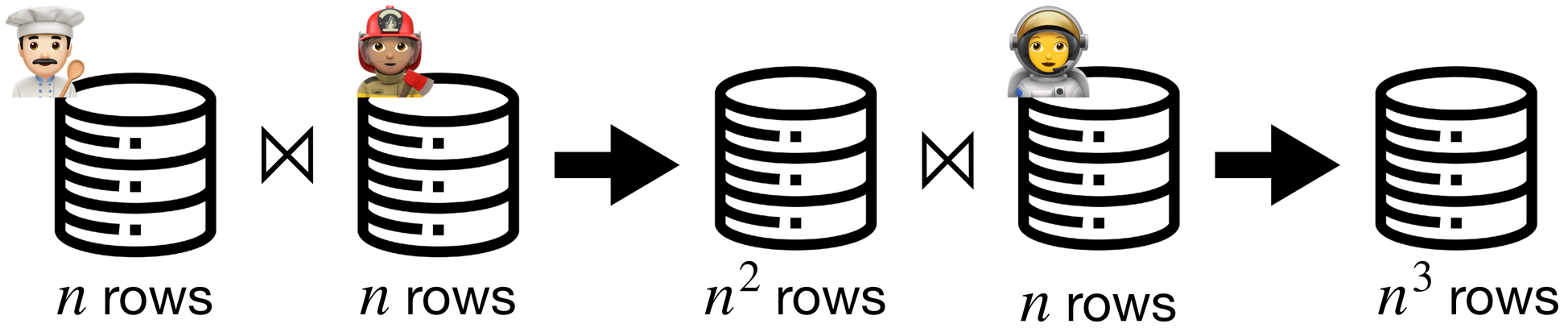
Oblivious Joins with Cartesian Product

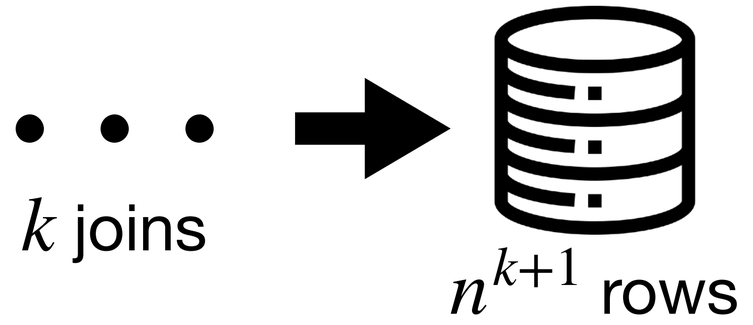
Cust \ Ord	🐸	🤡	🥶	🥶	🤡	🤡	😈
🤡	❌	✅	❌	❌	✅	✅	❌
😈	❌	❌	❌	❌	❌	❌	✅
🤡	❌	✅	❌	❌	✅	✅	❌
🐸	✅	❌	❌	❌	❌	❌	❌
🥶	❌	❌	✅	✅	❌	❌	❌
🥶	❌	❌	✅	✅	❌	❌	❌
🐸	✅	❌	❌	❌	❌	❌	❌

🔒

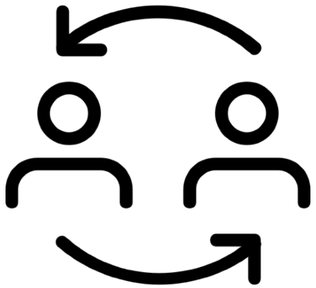
$O(n^2)$ work
$O(n^2)$ output size

The Problem


n rows
$\bowtie$
n rows
n^2 rows
$\bowtie$
n rows
n^3 rows


k joins
n^{k+1} rows

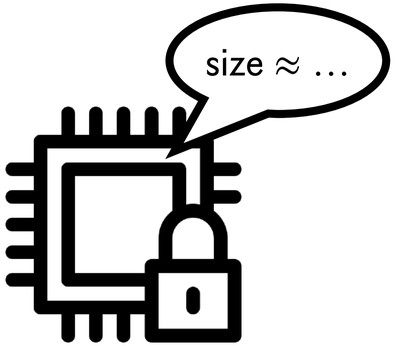
Prior Approaches to Collaborative Analytics



Peer-to-peer setting

e.g., Conclave (EuroSys'19),
Senate (Security'21), SMCQL (VLDB'17),
Shrinkwrap (VLDB'18)

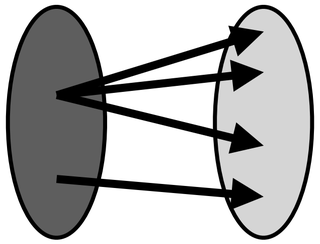
❌ Doesn't scale with more parties



Leakage or upper bounds

e.g., Opaque (NSDI'17), Obliviator (Security'25),
SAQE (VLDB'20)

❌ Only provides partial security



Individual join operators

e.g., [MRR] (CCS'20), [BDG+] (CCS'22),
[AHK+] (CCS'23)

❌ Cannot evaluate queries

Can we efficiently evaluate complex analytics without compromising security?

Yes!

We are the first to show **multiple joins** with no security compromises

…and run **all queries** from prior work, with **orders-of-magnitude** larger inputs

What do real-world queries look like?

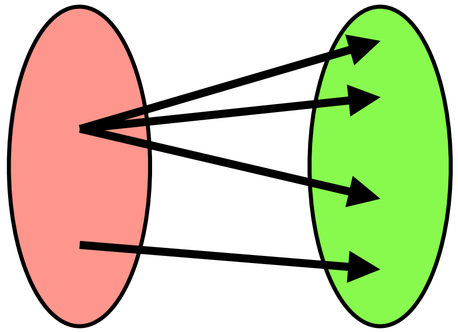
We collect all **31 queries** from related work in MPC analytics

All have output size bound $O(n)$ instead of $O(n^k)$!

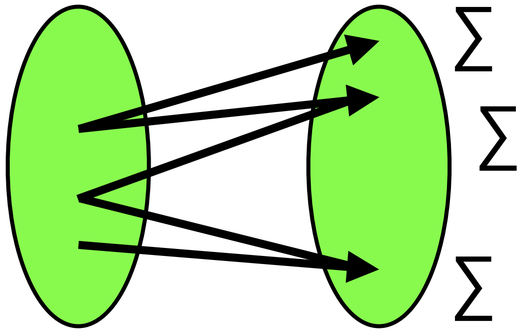
Why?

decomposable

one-to-many



many-to-many + aggregation

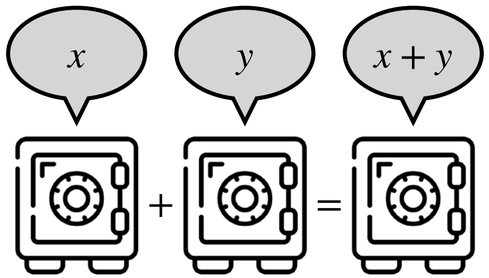


Our Insight

The general problem is impractically difficult

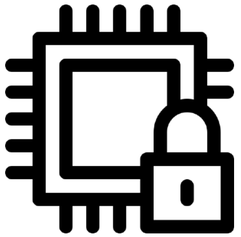
Practical instances are not!

Common query semantics impose an
$O(n)$ bound on the output



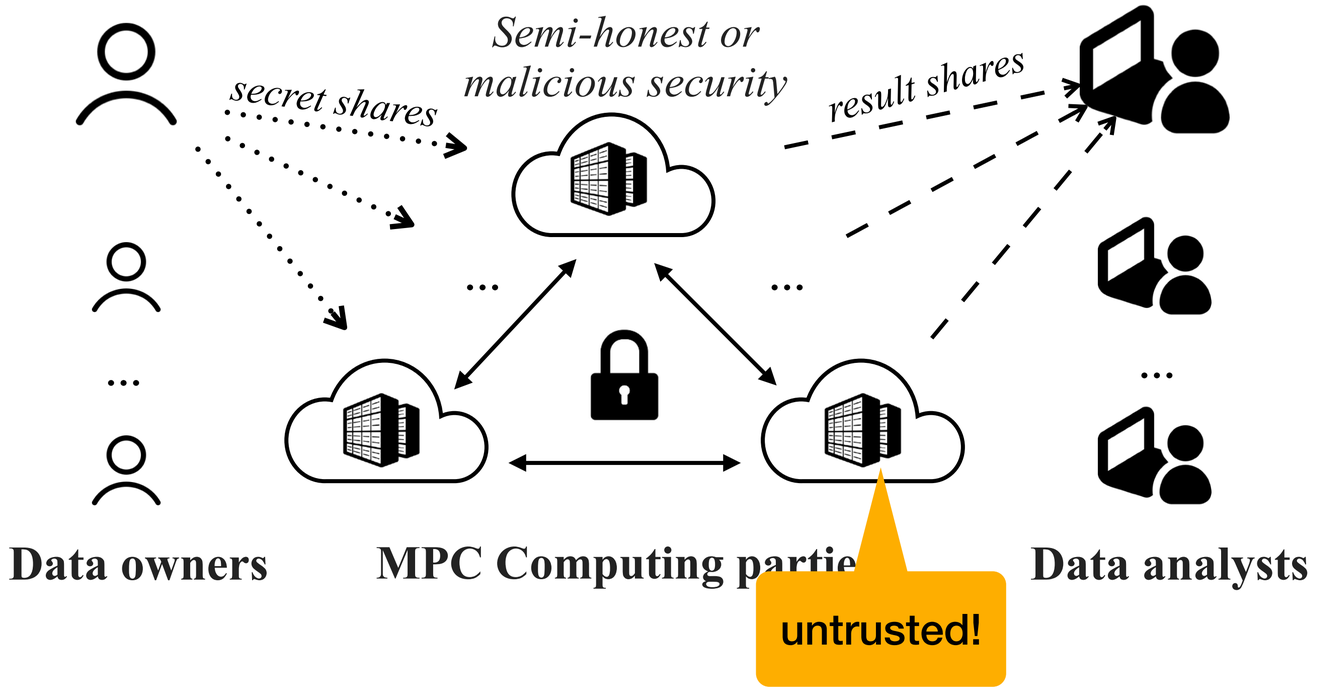
Decompose joins into sort + aggregation

Applies beyond multiparty computation!



Secure Multiparty Computation (MPC)

Outsourced Setting



Secret Sharing: Data Owners

ID	Balance
1	$100
4	$50
9	$25
7	-$13

ID	Balance
3	$814
2	$312
7	$60
6	-$366

ID	Balance
14	$82
12	$14
0	-$8
3	$160

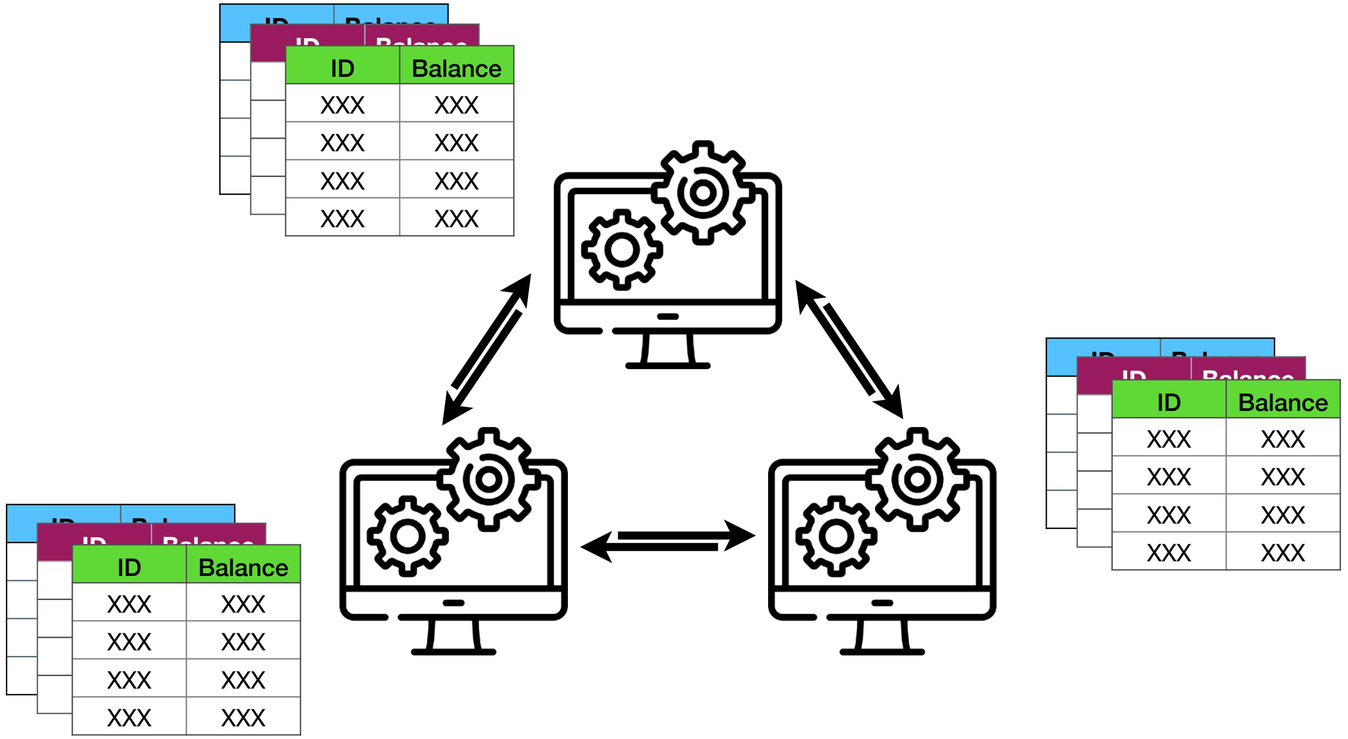
Secret Sharing: Data Owners

ID	Balance
1799JtRkU	NV9MVqRf
qicpdFpxn	ZgaoE7Hx
JtRkUmc5	FVwp7XTF
qSiJ3nBG	vhnCMIo7j

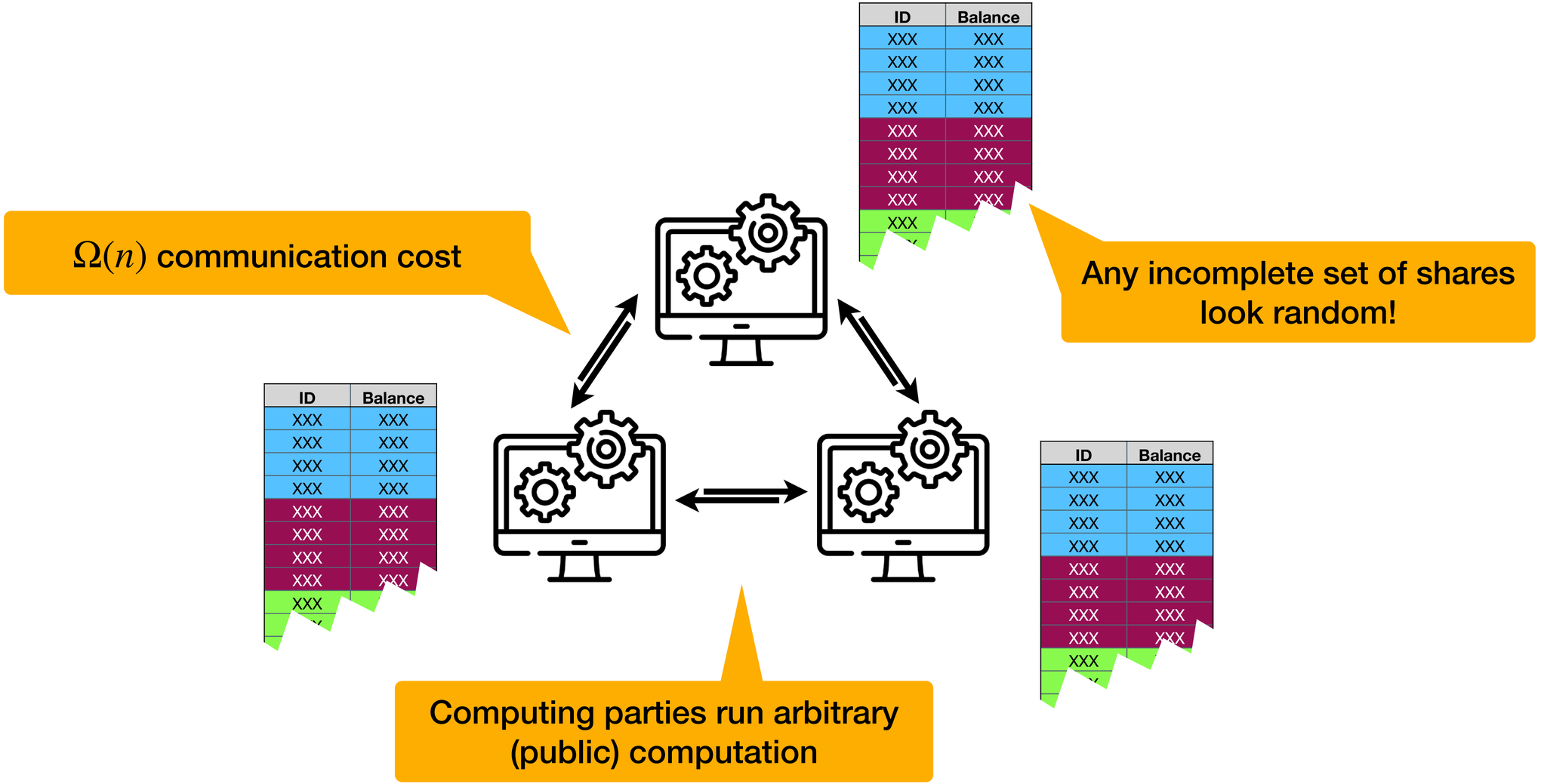
ID	Balance
UW6m9tx	EABqfgpRi
m6vQFTgb	9u19eWyV
qhq2JFbe	hM2hOg8
MQeNGgyI	q1uon8xM

ID	Balance
hsIcAgWLy	OUPgb2V
MVAJi4Vnt	FzI0j2hH8
8KsbVnN	YIaNhg5py
ehbzAtm6j	LmWp415

Secret Sharing: Computing Parties



Secret Sharing: Computing Parties



Secret Shared Tables 🔒

ID	Balance
1799JtRkU	NV9MVqRf
qicpdFpxn	ZgaoE7Hx
JtRkUmc5	FVwp7XTF
qSiJ3nBG	vhnCMIo7j

ID	Balance
UW6m9tx	EABqfgpRi
m6vQFTgb	9u19eWyV
qhq2JFbe	hM2hOg8
MQeNGgyI	q1uon8xM

ID	Balance
hsIcAgWLy	OUPgb2V
MVAJi4Vnt	FzI0j2hH8
8KsbVnN	YIaNhg5py
ehbzAtm6j	LmWp415

Secret Shared Tables 🔒

ID	Balance
1	$100
4	$50
9	$25
7	-$13

ID	Balance
3	$814
2	$312
7	$60
6	-$366

ID	Balance
14	$82
12	$14
0	-$8
3	$160

Improving on Quadratic: Sort & Aggregate

One table must have unique keys

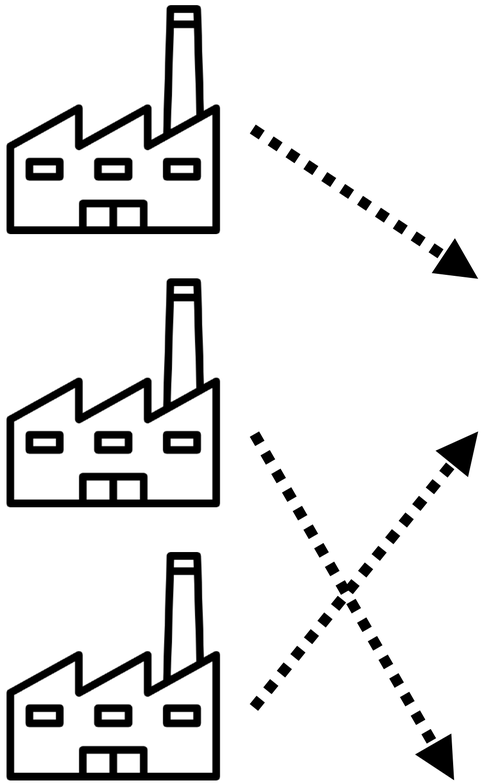
$O(n \log n)$

```
func join(L, R, keys):
   T := concat*(L, R)
   T.sort(keys)
   T.prefix_aggregate(…)
   return T
```

$O(n \log n)$

Extends to **inner, outer, anti,** and **semi** joins.

Example with Our Approach



Customers

Customer	AcctBalance
🤡	$1000
😈	$2000
🤡	$500
🐸	$60
🥶	$8000
🥶	$700
🐸	-$200

⋈

Orders

Customer	OrdCost
🐸	$30
🤡	$20
🥶	$55
🥶	$18
🤡	$70
🤡	$12
😈	$8

duplicates!

Example with Our Approach

```
SELECT
    SPECIES, SUM(ACCTBALANCE) / SUM(ORDCOST)
FROM
    ORDERS, CUSTOMERS
WHERE
    ORDER.CUSTOMER = CUSTOMERS.CUSTOMER
GROUP BY
    SPECIES
```

Per customer group, what is the ratio of account balance to order total?

Customers

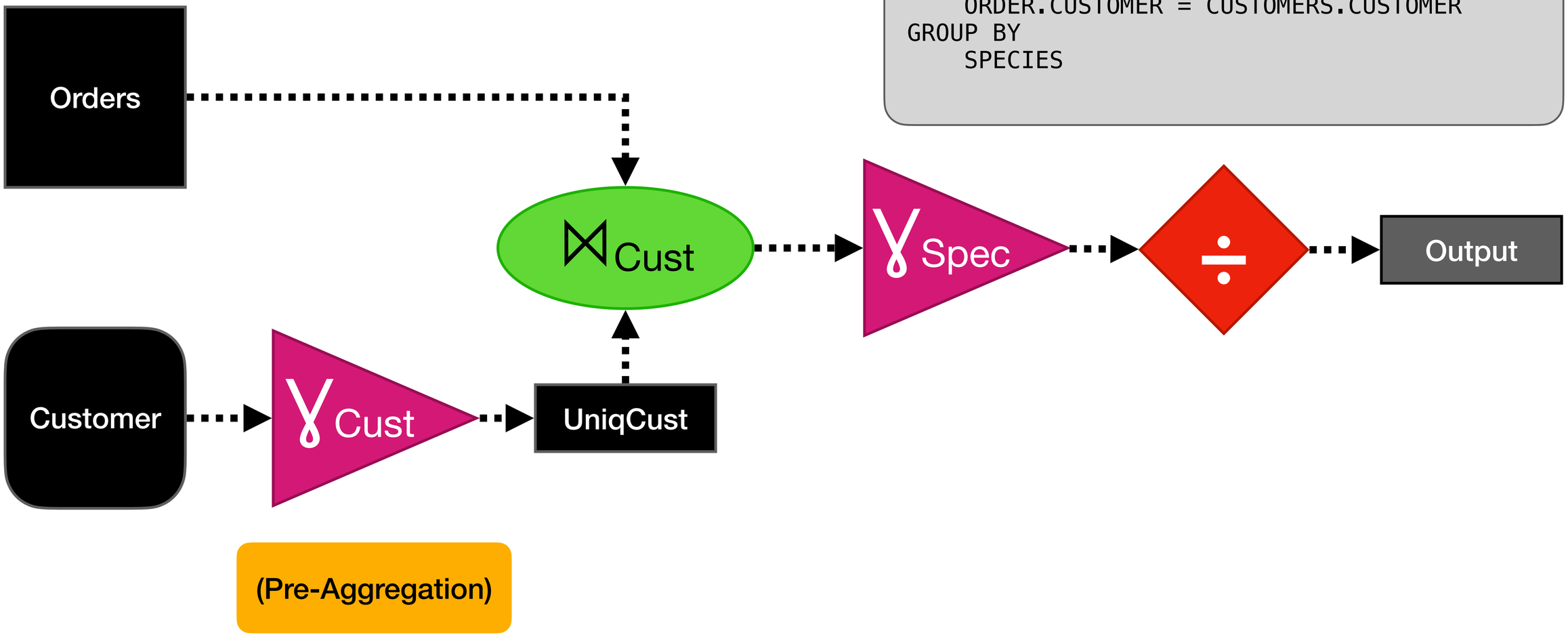
Customer	AcctBalance
🤡	$1000
😈	$2000
🤡	$500
🐸	$60
🥶	$8000
🥶	$700
🐸	-$200

Orders

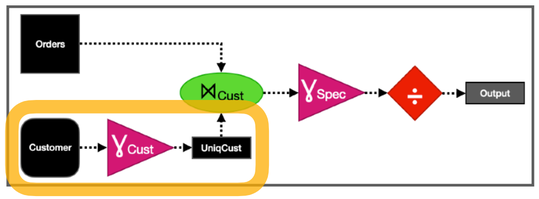
Customer	OrdCost
🐸	$30
🤡	$20
🥶	$55
🥶	$18
🤡	$70
🤡	$12
😈	$8

Query Plan

```
SELECT
    SPECIES, SUM(ACCTBALANCE) / SUM(ORDCOST)
FROM
    ORDERS, CUSTOMERS
WHERE
    ORDER.CUSTOMER = CUSTOMERS.CUSTOMER
GROUP BY
    SPECIES
```



Pre-Aggregate



Customers

Customer	AcctBalance
🤡	$1000
😈	$2000
🤡	$500
🐸	$60
🥶	$8000
🥶	$700
🐸	-$200

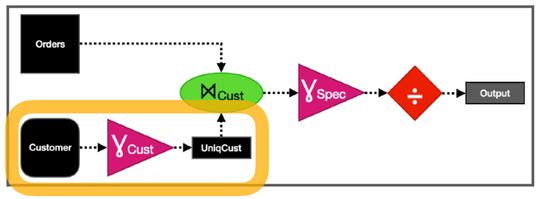
Orders

Customer	OrdCost
🐸	$30
🤡	$20
🥶	$55
🥶	$18
🤡	$70
🤡	$12
😈	$8

Pre-Aggregate

Customers

Customer	AcctBalance
🤡	$1000
😈	$2000
🤡	$500
🐸	$60
🥶	$8000
🥶	$700
🐸	-$200

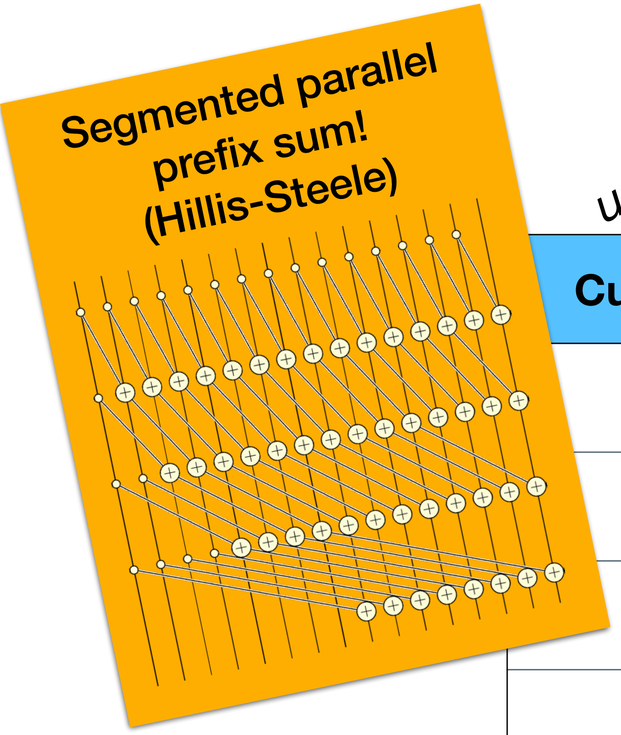


Orders

Customer	OrdCost
🐸	$30
🤡	$20
🥶	$55
🥶	$18
🤡	$70
🤡	$12
😈	$8

After Pre-aggregation

One-to-many join



unique keys!

Customers

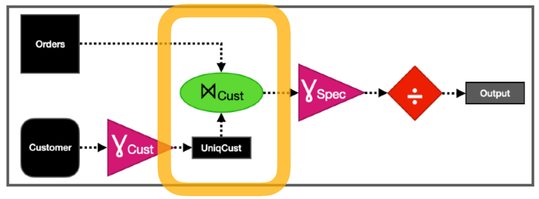
Customer	TotalAcctBalance
🤡	$1500
😈	$2000
🐸	-$140
🥶	$8700

pre-aggregated values

⋈

Orders

Customer	OrdCost
🐸	$30
🤡	$20
🥶	$55
🥶	$18
🤡	$70
🤡	$12
😈	$8



After Pre-aggregation

One-to-many join

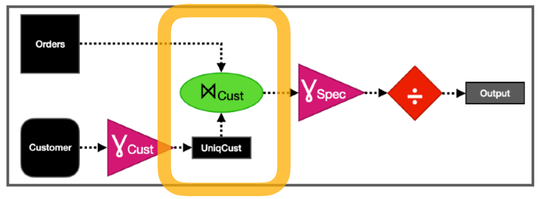
Customers

Customer	TotalAcctBalance
🤡	$1500
😈	$2000
🐸	-$140
🥶	$8700

⋈

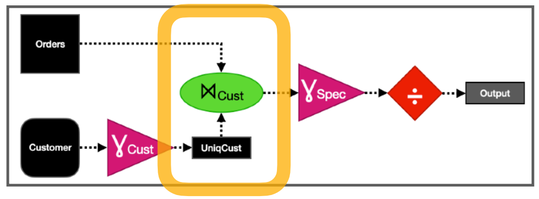
Orders

Customer	OrdCost
🐸	$30
🤡	$20
🥶	$55
🥶	$18
🤡	$70
🤡	$12
😈	$8



Concatenate

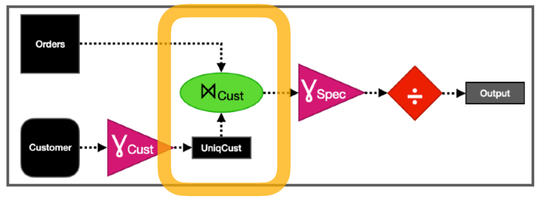
```
func join(L, R, keys):
    T := concat*(L, R)
    T.sort(keys)
    T.prefix_aggregate(…)
    return T
```



Customer	C.Total	O.Cost
🤡	$1500	
😈	$2000	
🐸	-$140	
🥶	$8700	
🐸		$30
🤡		$20
🥶		$55
🥶		$18
🤡		$70
🤡		$12
😈		$8

Sort

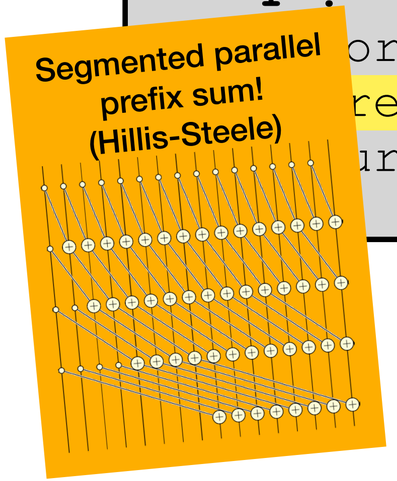
```
func join(L, R, keys):
    T := concat*(L, R)
    T.sort(keys)
    T.prefix_aggregate(…)
    return T
```

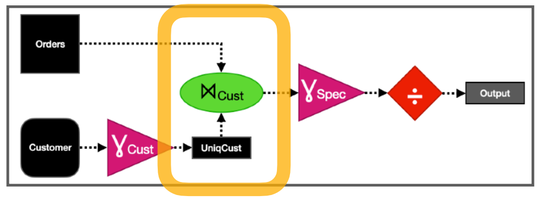


Customer	C.Balance	O.Cost
🤡	$1500	
🤡		$20
🤡		$70
🤡		$12
😈	$2000	
😈		$8
🐸	-$140	
🐸		$30
🥶	$8700	
🥶		$55
🥶		$18

Group Aggregate

```
func join(L, R, keys):
    T := concat*(L, R)
    T.sort(keys)
    T.prefix_aggregate(…)
    return T
```

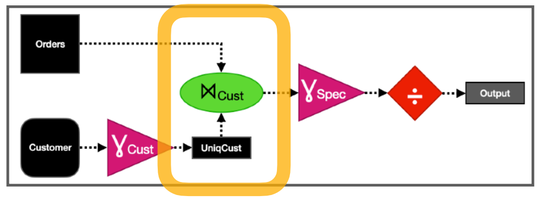




Customer	C.Balance	O.Cost	TotalCost
🤡	$1500	∅	
🤡	$1500	$20	
🤡	$1500	$70	
🤡	$1500	$12	$102
😈	$2000	∅	
😈	$2000	$8	$8
🐸	-$140	∅	
🐸	-$140	$30	$30
🥶	$8700	∅	
🥶	$8700	$55	
🥶	$8700	$18	$73

Mark Invalid Rows

```
func join(L, R, keys):
    T := concat*(L, R)
    T.sort(keys)
    T.prefix_aggregate(…)
    T.mark_invalid()
    return T
```

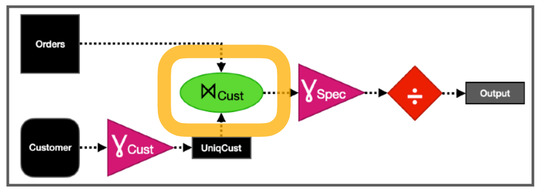


Customer	C.Balance	O.Cost	TotalCost
∅	∅	∅	∅
∅	∅	∅	∅
∅	∅	∅	∅
🤡	$1500	$12	$102
∅	∅	∅	∅
😈	$2000	$8	$8
∅	∅	∅	∅
🐸	-$140	$30	$30
∅	∅	∅	∅
∅	∅	∅	∅
🥶	$8700	$18	$73

Join Output

```
SELECT
    SPECIES, SUM(ACCTBALANCE) / SUM(ORDCOST)
FROM
    ORDERS, CUSTOMERS
WHERE
    ORDER.CUSTOMER = CUSTOMERS.CUSTOMER
GROUP BY
    SPECIES
```

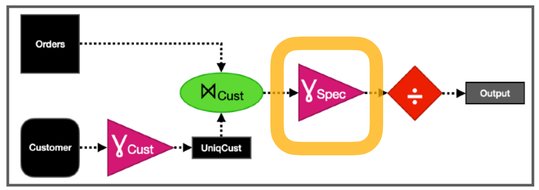
Customer	C.Balance	TotalCost	Species
🤡	$1500	$102	Human
🥶	$8700	$73	Human
😈	$2000	$8	NonHuman
🐸	-$140	$30	NonHuman



Final Aggregation

```
SELECT
    SPECIES, SUM(ACCTBALANCE) / SUM(ORDCOST)
FROM
    ORDERS, CUSTOMERS
WHERE
    ORDER.CUSTOMER = CUSTOMERS.CUSTOMER
GROUP BY
    SPECIES
```

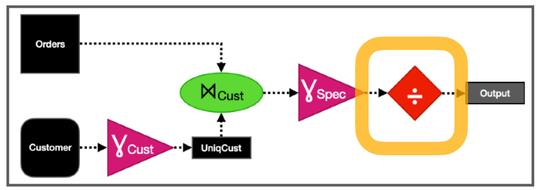
Species	Customer	C.Balance	TotalCost
Human	∅	∅	∅
Human	∅	$10,200	$175
NonHuman	∅	∅	∅
NonHuman	∅	$1,860	$38



Compute Ratio

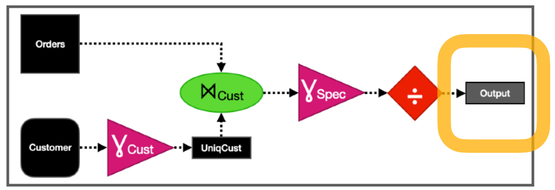
```
SELECT
    SPECIES, SUM(ACCTBALANCE) / SUM(ORDCOST)
FROM
    ORDERS, CUSTOMERS
WHERE
    ORDER.CUSTOMER = CUSTOMERS.CUSTOMER
GROUP BY
    SPECIES
```

Species	C.Balance	TotalCost	Ratio
Human	$10,200	$175	**58**
NonHuman	$1,860	$38	**48**



Final Table

```
SELECT
    SPECIES, SUM(ACCTBALANCE) / SUM(ORDCOST)
FROM
    ORDERS, CUSTOMERS
WHERE
    ORDER.CUSTOMER = CUSTOMERS.CUSTOMER
GROUP BY
    SPECIES
```



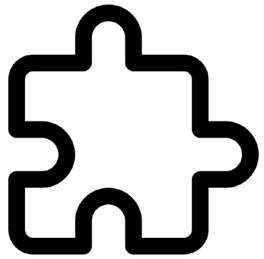
Still secret shared! More joins…

Species	Ratio
Human	58
NonHuman	48

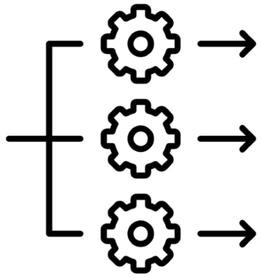
Oblivious output bound:
|Orders|

ORQ: an Oblivious Relational Query engine

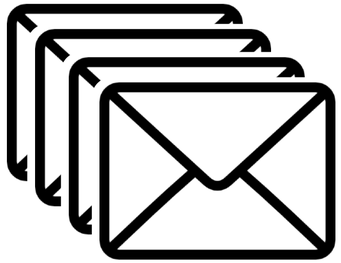
Familiar APIs &
simple orchestration



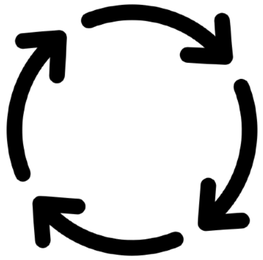
Support for multiple
protocols & threat models



Data-parallel runtime



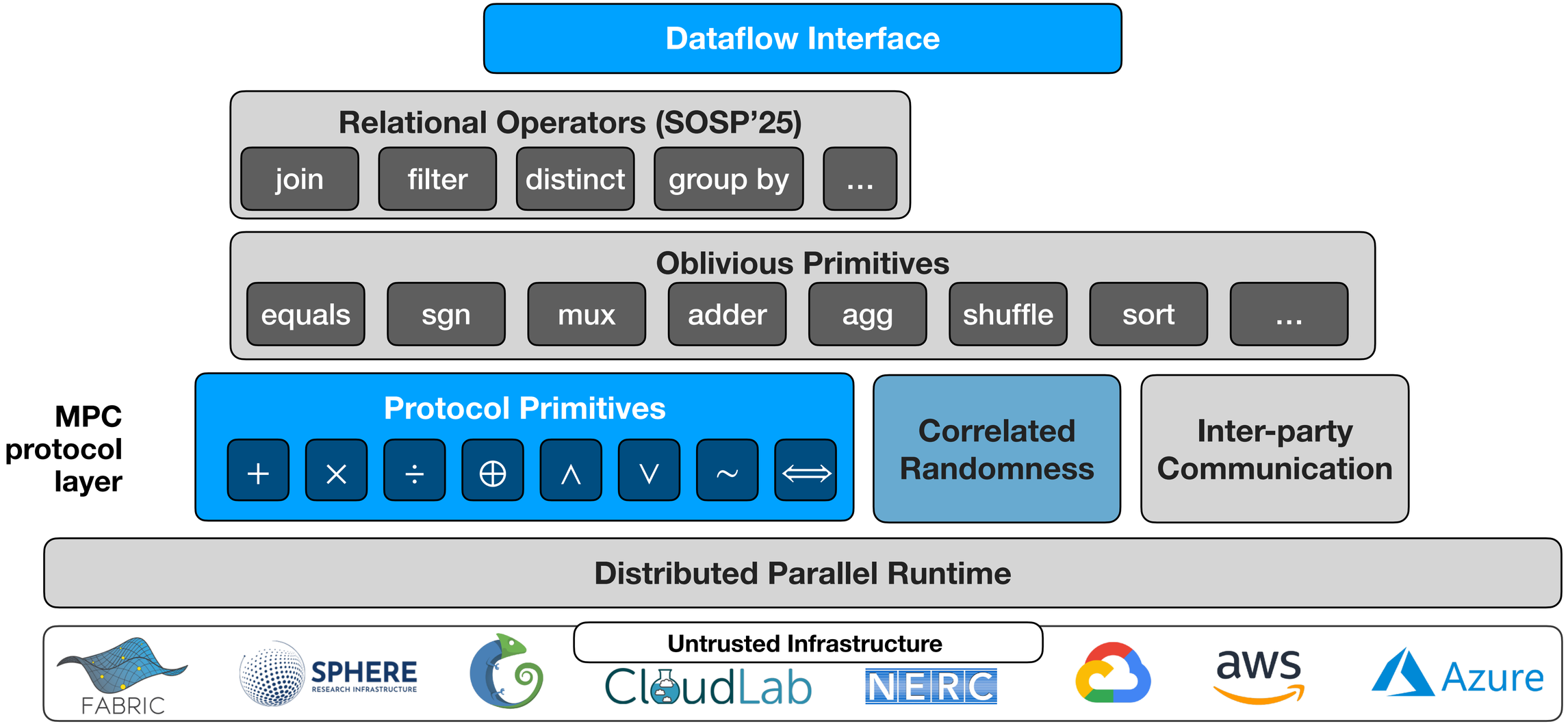
Vectorization &
message batching



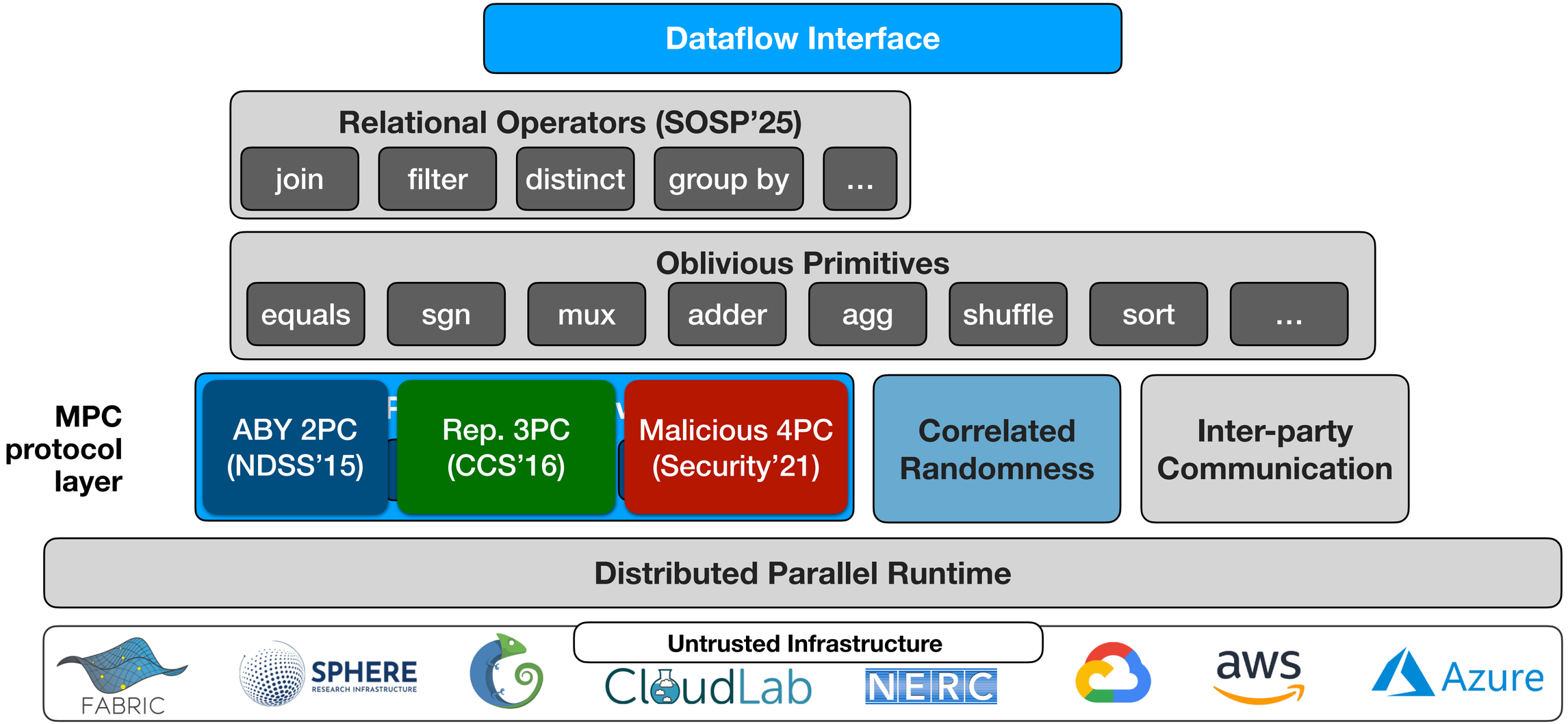
Custom TCP communicator

14,000 C++ LoC

ORQ Stack


Dataflow Interface
Relational Operators (SOSP'25)
join
filter
distinct
group by
...
Oblivious Primitives
equals
sgn
mux
adder
agg
shuffle
sort
...
MPC protocol layer
Protocol Primitives
+
×
÷
⊕
∧
∨
~
⟺
Correlated Randomness
Inter-party Communication
Distributed Parallel Runtime
Untrusted Infrastructure
FABRIC
SPHERE
RESEARCH INFRASTRUCTURE
CloudLab
NERC
aws
Azure

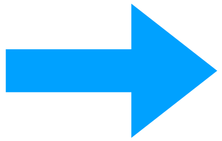
ORQ Stack


Dataflow Interface
Relational Operators (SOSP'25)
join
filter
distinct
group by
...
Oblivious Primitives
equals
sgn
mux
adder
agg
shuffle
sort
...
MPC protocol layer
ABY 2PC (NDSS'15)
Rep. 3PC (CCS'16)
Malicious 4PC (Security'21)
Correlated Randomness
Inter-party Communication
Distributed Parallel Runtime
Untrusted Infrastructure
FABRIC
SPHERE
RESEARCH INFRASTRUCTURE
CloudLab
NERC
aws
Azure

API Example

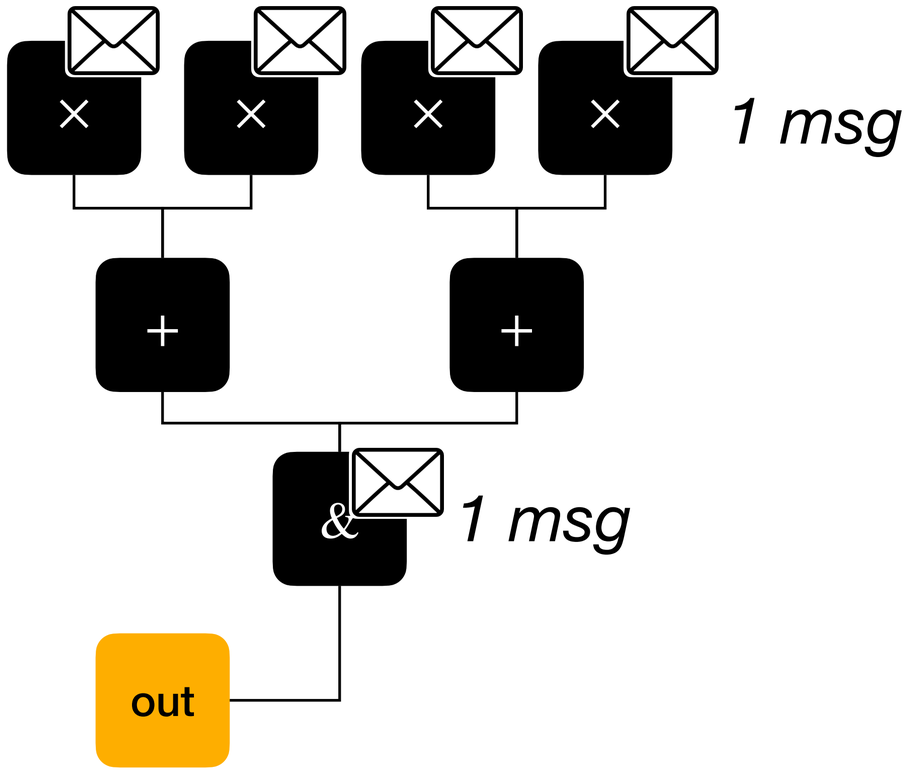
TPC-H Q13: SQL vs. ORQ Dataflow API

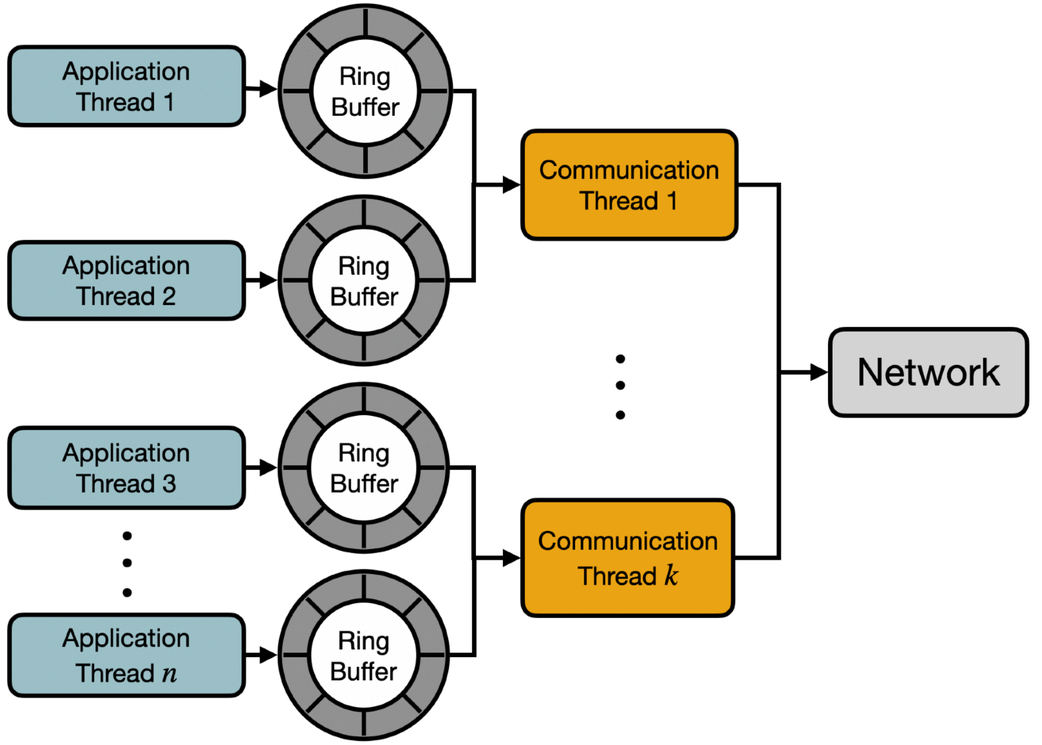
```
select
    c_count, count(*) as custdist
from (
    select
        c_custkey,
        count(o_orderkey)
    from
        customer left outer join orders on
            c_custkey = o_custkey
            and o_comment != 0
    group by
        c_custkey
    ) as c_orders (c_custkey, c_count)
group by
    c_count
order by
    custdist desc,
    c_count desc;
```



```
auto C = db.getCustomersTable();
auto O = db.getOrdersTable();

O.filter(O["[Comment]"] != 0);

auto T = C.left_outer_join(O,
    {"[CustKey]"},
    {{"Count", "Count", count}});

T.convert_a2b("Count", “[Count]");

auto F = T.aggregate(
    {"[Count]"},
    {{"CustDist", "CustDist", count}});

F.convert_a2b("CustDist", "[CustDist]");
F.sort({"[CustDist]", "[Count]"}, DESC);
```

Vectorization & Message Batching





Flexible TCP Communicator

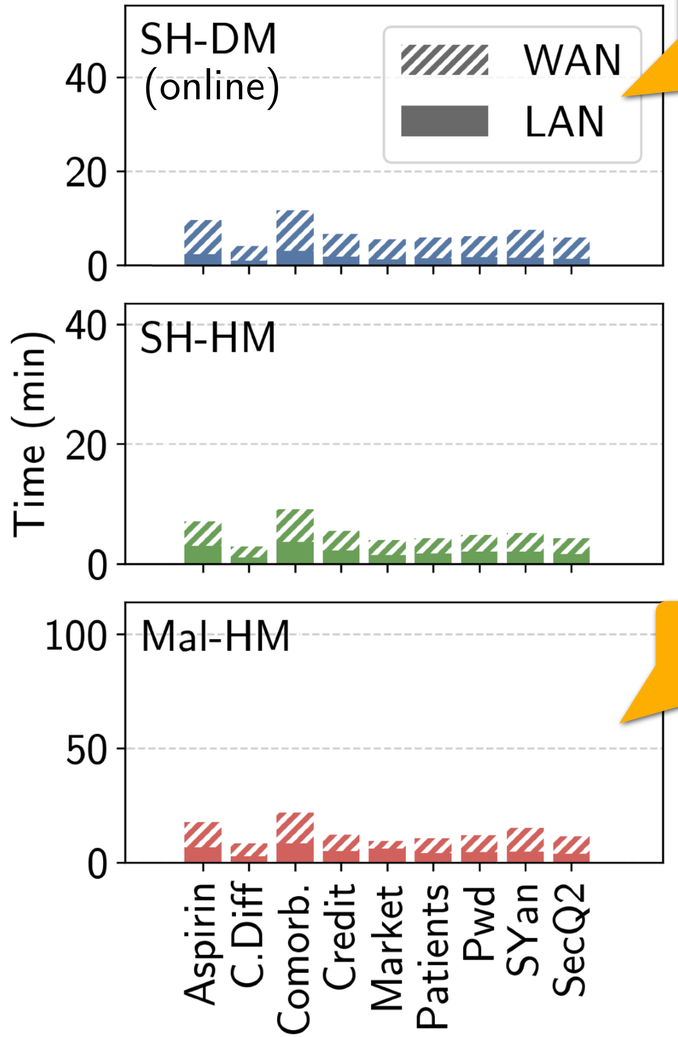
Prior MPC Benchmarks

with ~5M input rows

ABY 2PC
Semihonest / Dishonest Majority

Replicated 3PC
Semihonest / Honest Majority

Fantastic 4PC
Malicious / Honest Majority



6 Gbps 20 ms
25 Gbps 0.3 ms

all ≤ 3 min

all ≤ 4 min

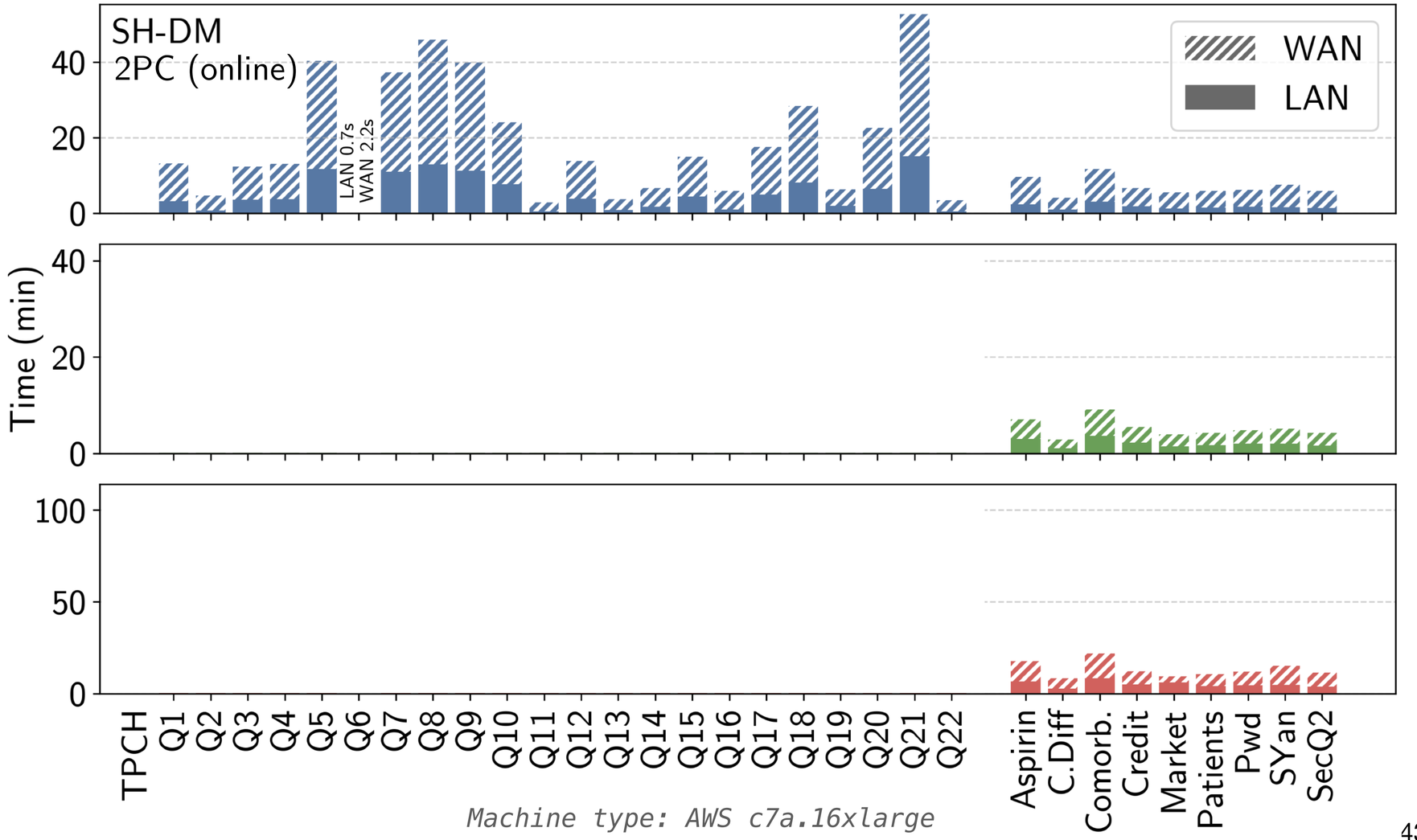
Motivation for leakage!

all ≤ 9 min

Full TPC-H Benchmark under MPC

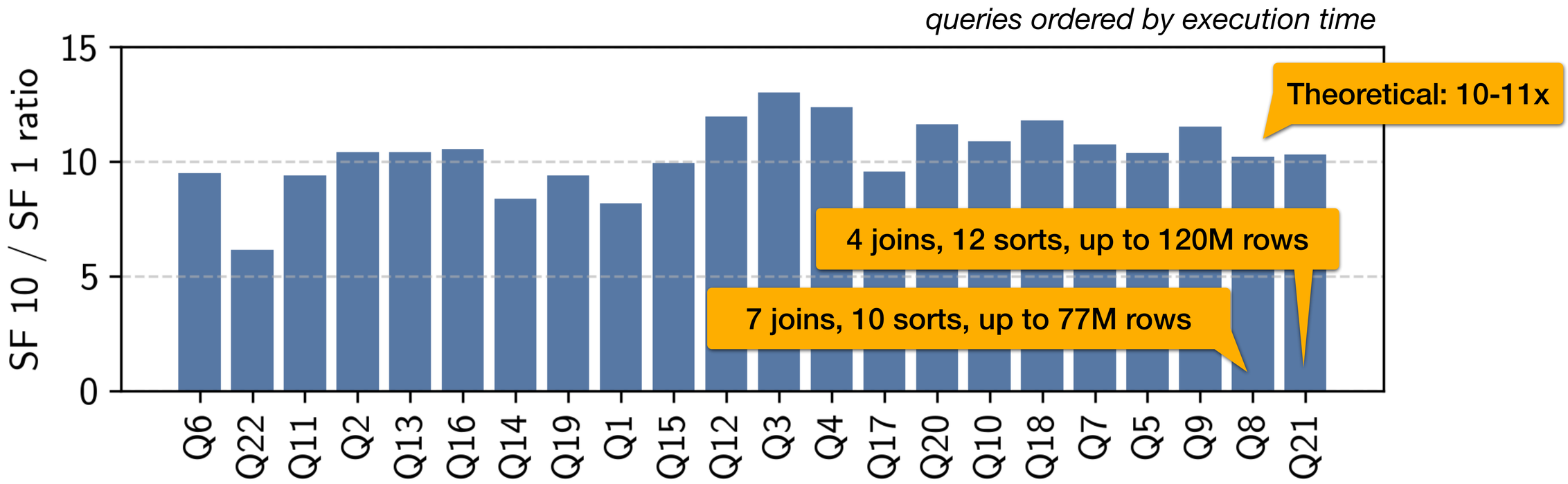
@ Scale Factor 1 (≈ 1 GB input)

	TPC-H	
minutes	Median	Max
SH-DM LAN	3.8	15.0
WAN	13.5	52.7



Scalability

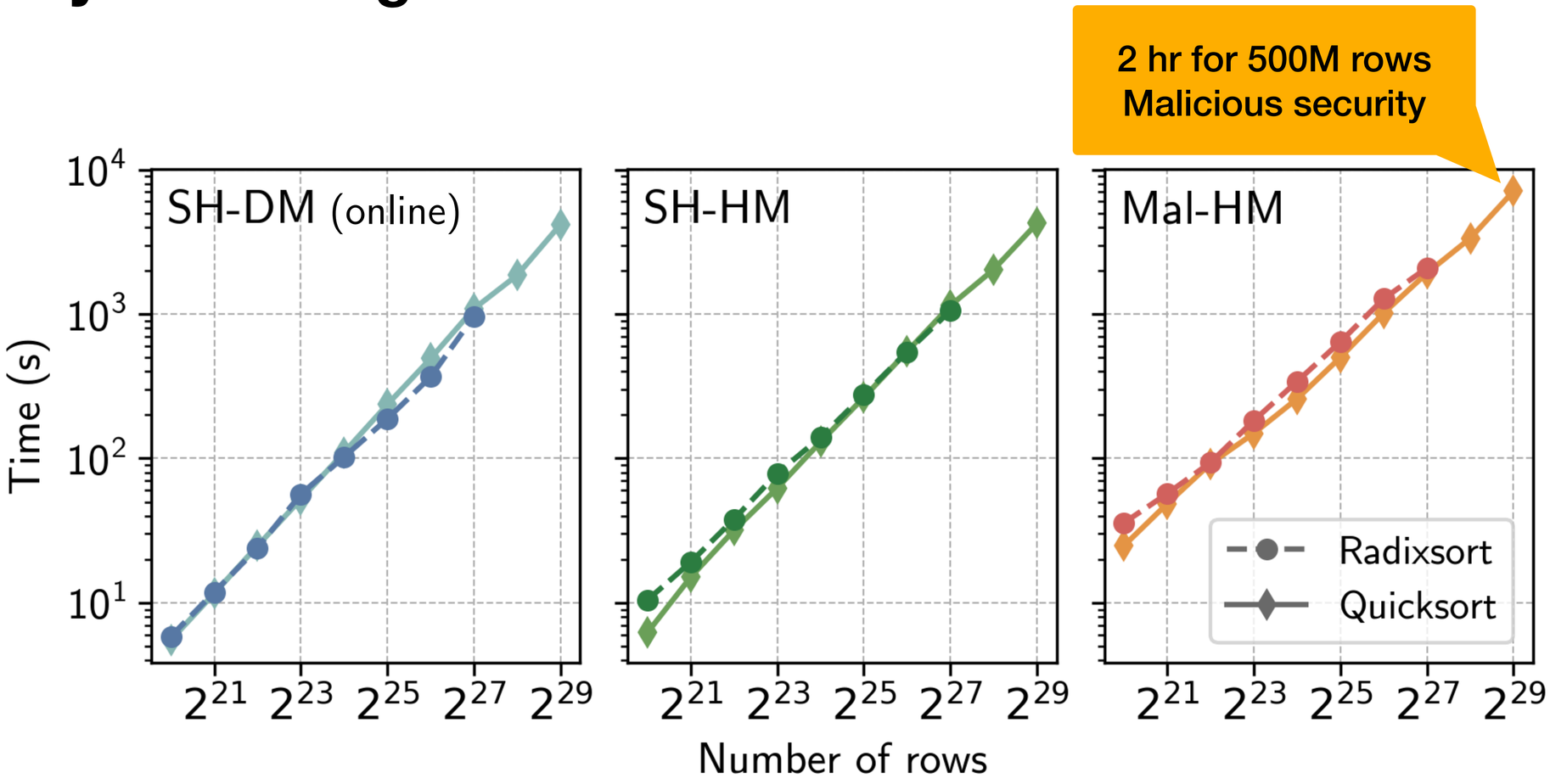
10x larger inputs (SF 10)



Machine type: AWS c7a.16xlarge

Going Even Further?

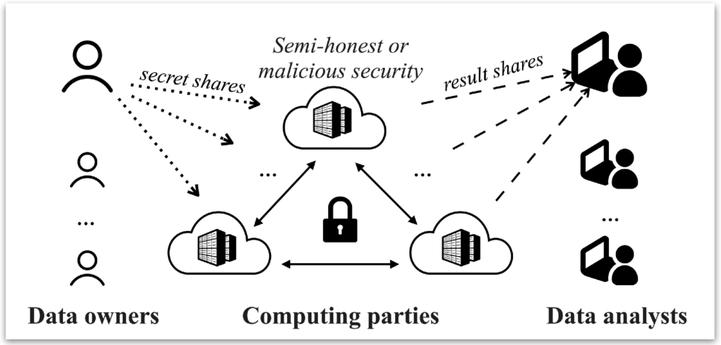
Scalability of Sorting



Machine type: AWS c7a.16xlarge

ORQ: Complex Analytics on Private Data with Strong Security Guarantees *(SOSP 2025)*

email me: elibaum@bu.edu



BU

CASPLab

We're hiring!

Paper/Code/Docs

elibaum.com

Privacy

Runs on untrusted compute with end-to-end oblivious execution

Generality

Uses crypto protocols as a blackbox to meet different security requirements

Expressivity

Efficiently computes all queries from prior work & TPC-H, with no leakage

Scalability

Evaluates queries with millions of rows & efficiently uses available resources

icons from flaticon.com

Backup

Future Work

- In progress:
  - Machine learning operators
  - Graph workloads
  - Cost modeling & SQL planner
- Explore other classes of queries:
  - Cyclic (`SELECT ... FROM A,B,C WHERE A.X=B.Y AND B.Y=C.W AND C.W=A.Z`)
  - Subclasses of non-decomposable functions (median, count distinct?)

Future Work

- Current implementation relies on trusted setup over SSH
  - Unified setup for mutually distrusting parties?
- We show $O(n\ell \cdot \log n \log \ell)$ communication, but $O(n\ell \cdot \ell)$ possible
- Better datatype support (currently: only int)
- Improved sorting protocols
- Scaling to multiple nodes per logical party

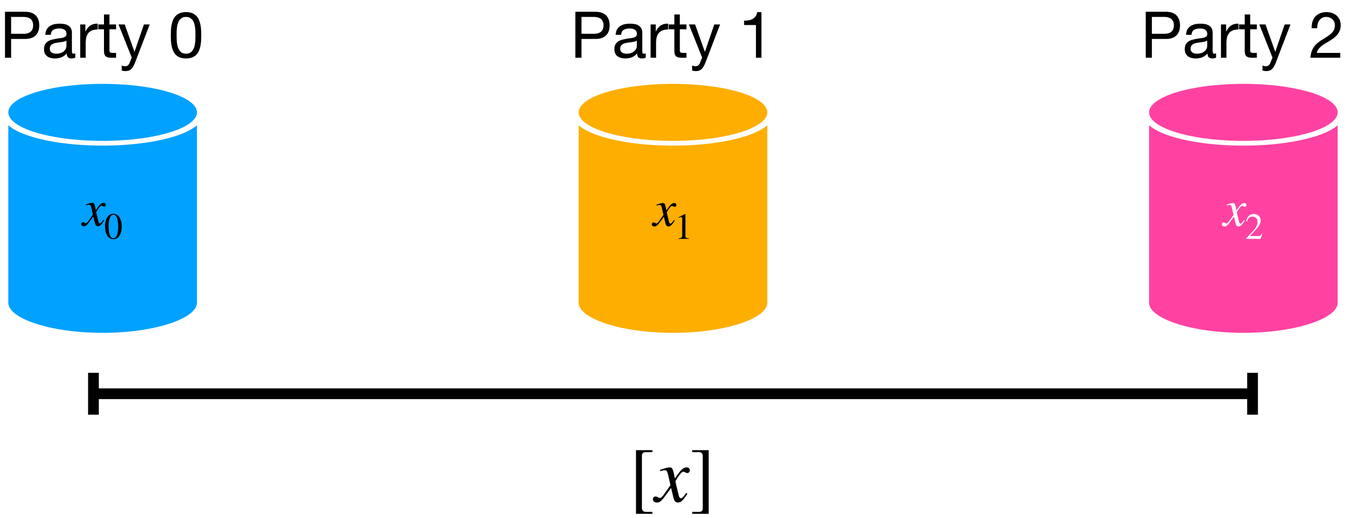
Quick Background on MPC

Secret-Sharing-Based MPC

secret

x $= x_0 + x_1 + x_2$

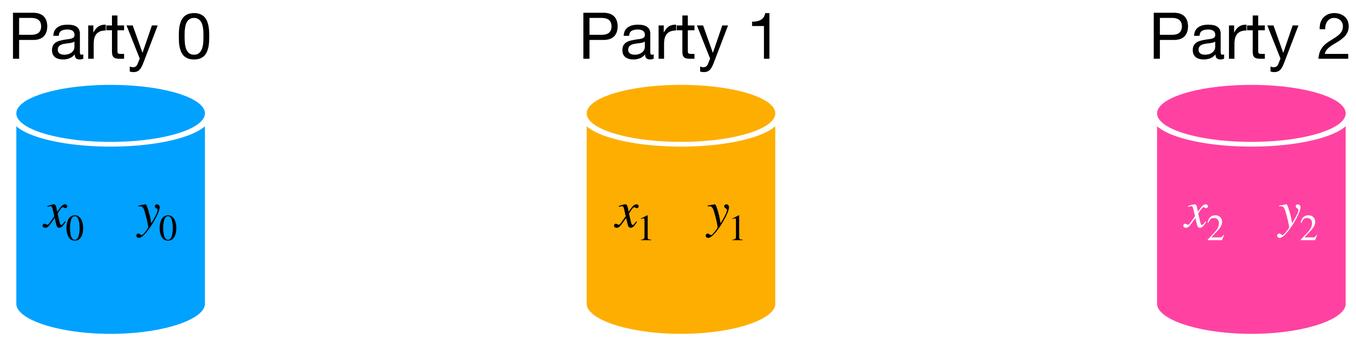
random



Arithmetic under MPC

Addition

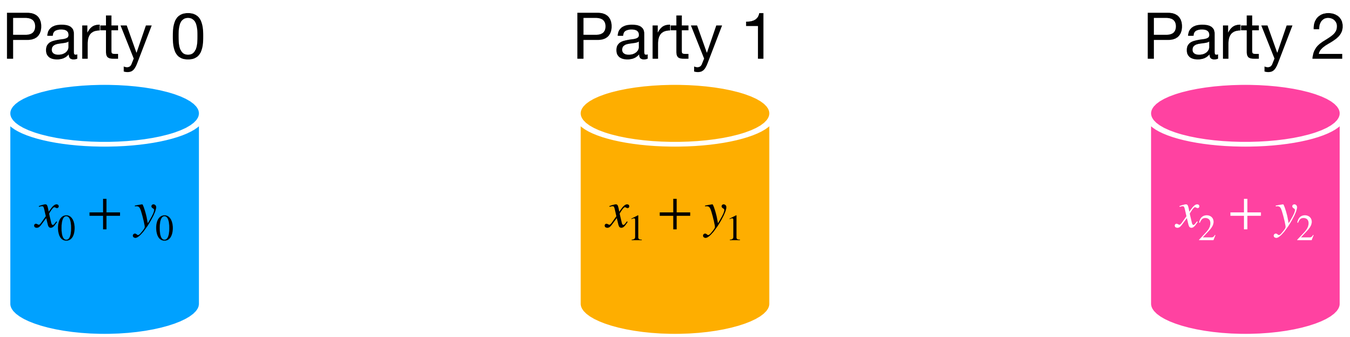
$$[s] = [x + y] = [x] + [y]$$



Arithmetic under MPC

Addition

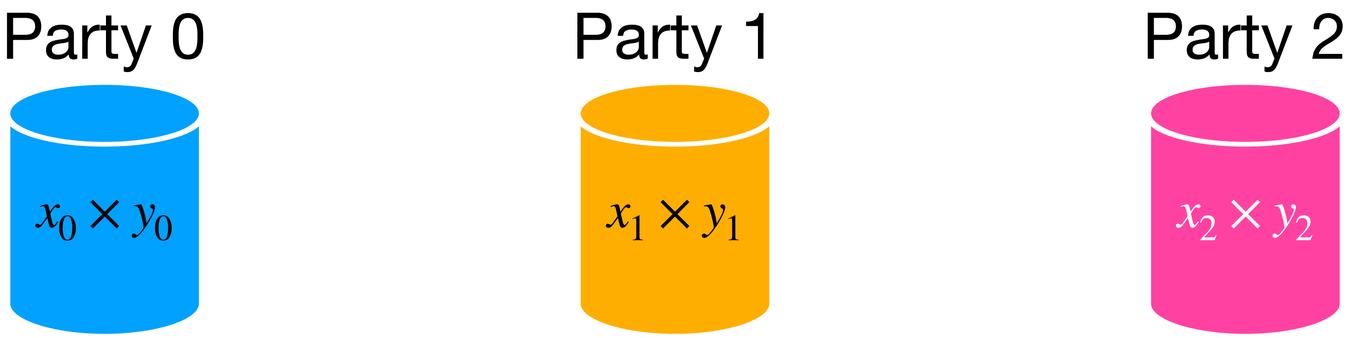
$$[s] = [x + y] = [x] + [y]$$



Arithmetic under MPC

Multiplication

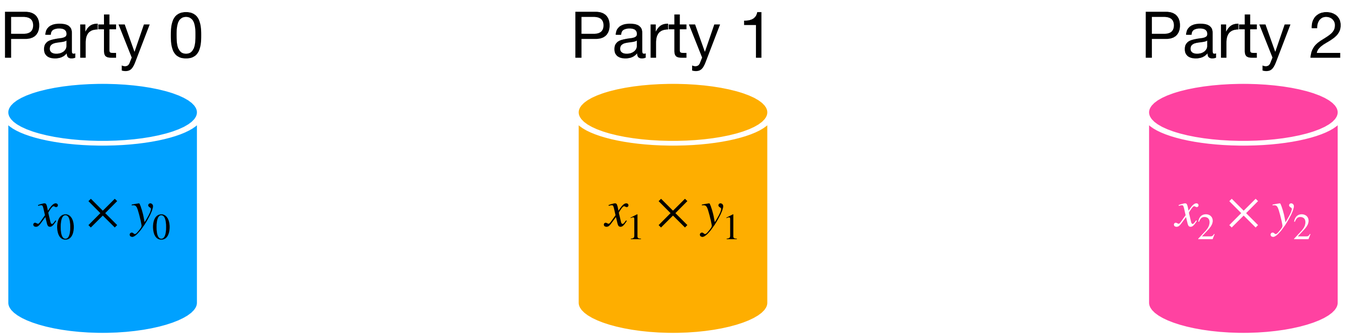
$$[p] = [x \times y] \stackrel{?}{=} [x] \times [y]$$



Arithmetic under MPC

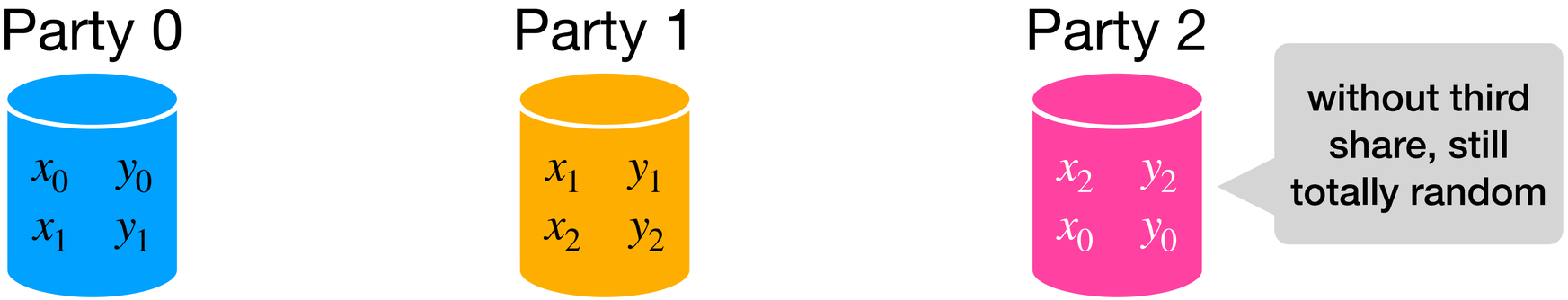
Multiplication

$$[p] = [x \times y] \neq [x] \times [y]$$



Arithmetic under MPC

Replicated Secret Sharing

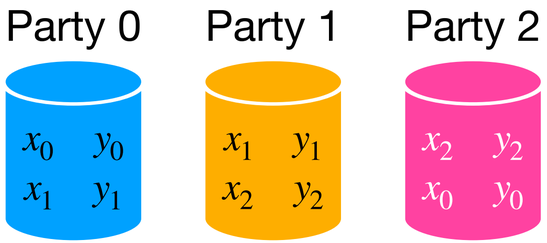


Arithmetic under MPC

Replicated Multiplication

$$[p] = [x \times y] = [(x_0 + x_1 + x_2)(y_0 + y_1 + y_2)] =$$

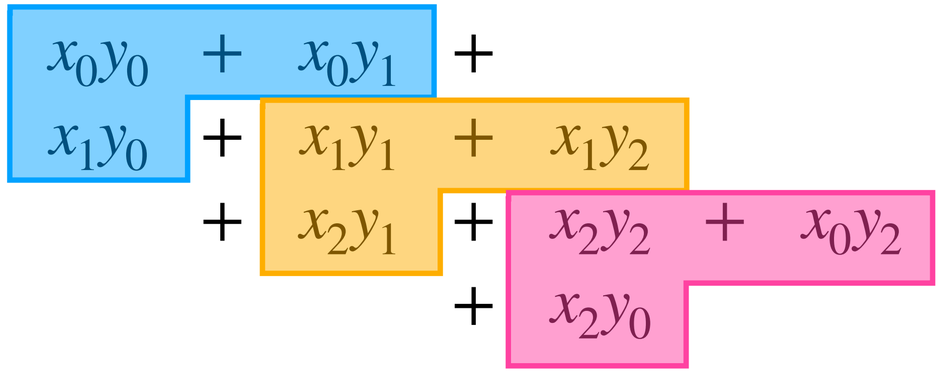
$$\begin{array}{ccccc} x_0y_0 & + & x_0y_1 & + & x_0y_2 \\ x_1y_0 & + & x_1y_1 & + & x_1y_2 \\ x_2y_0 & + & x_2y_1 & + & x_2y_2 \end{array}$$

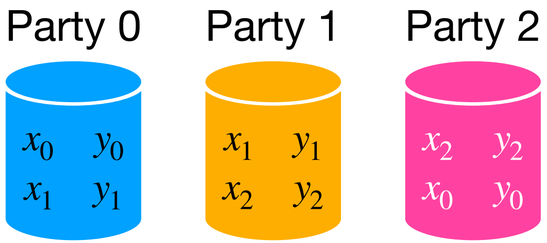


Arithmetic under MPC

Replicated Multiplication

$$[p] = [x \times y] = [(x_0 + x_1 + x_2)(y_0 + y_1 + y_2)] =$$

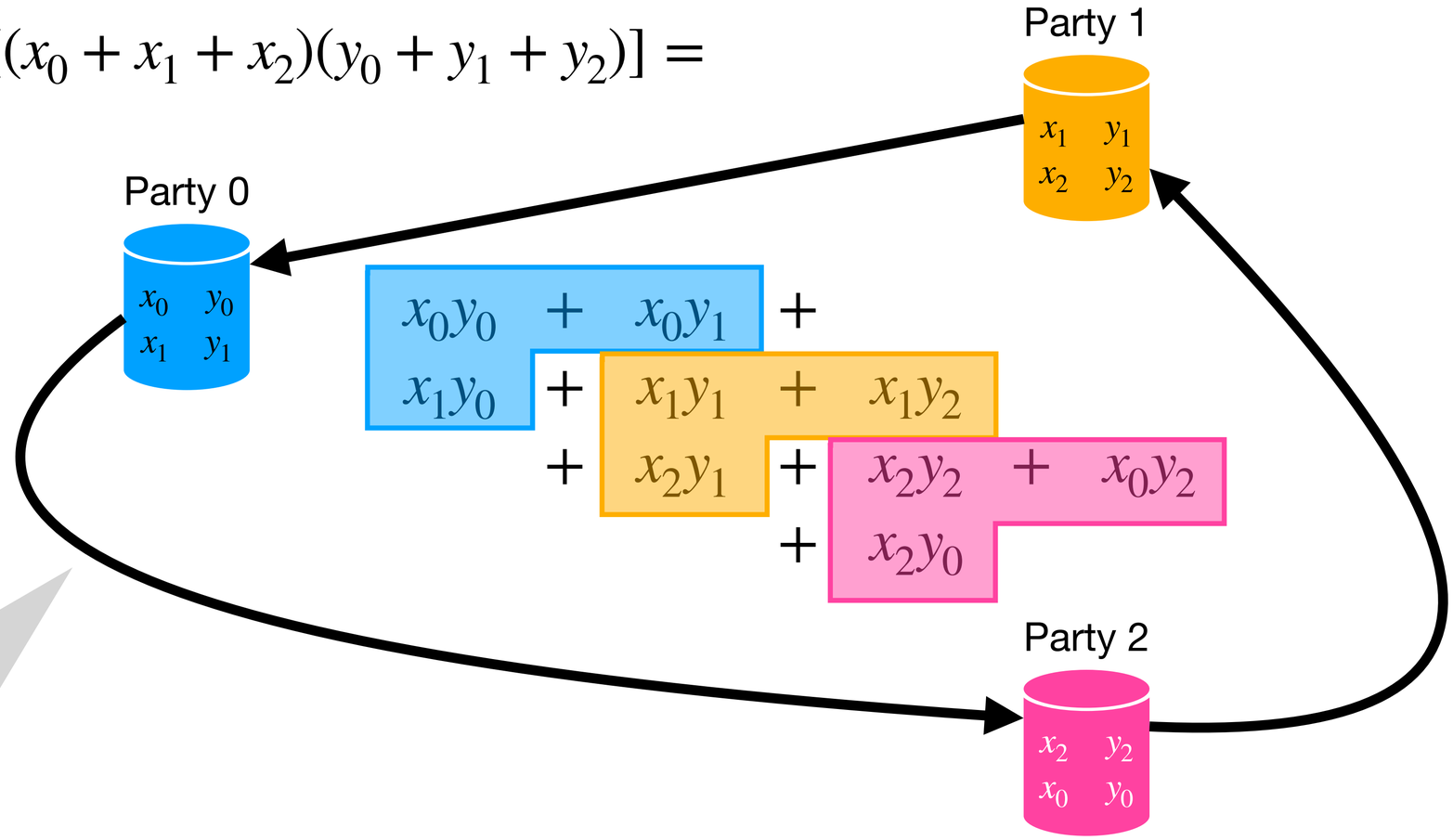




Arithmetic under MPC

Replicated Multiplication

$$[p] = [x \times y] = [(x_0 + x_1 + x_2)(y_0 + y_1 + y_2)] =$$



multiplication requires communication

Other Operations

- Equivalent **boolean** protocols for $\wedge$, $\oplus$, $\vee$ $\Longrightarrow$ circuits for:
  - Ripple-carry and parallel-prefix addition
  - Comparisons
  - Sorting
  - Integer division
  - Parallel aggregation networks
  - *Conversions* between share types

each AND gate requires communication!

What makes MPC expensive?

- ℓ-bit primitives (AND, MULT) require $\approx \ell$ bits of communication per party
- Decomposition of complex queries non-trivial, but for input size n,

$$O(n \log n \cdot \ell \log \ell) \text{ communication}$$

- Concretely: most expensive query (TPC-H Q21)

1 GB input → 50 GB comm/party

Adding a New Protocol

ORQ is designed to be extensible:

- Define the **secret sharing** scheme & **threat model**
- Implement the **multiplication primitive** & other low-level operations
- Configure **communication**, and **correlated randomness**, if needed

ORQ is protocol agnostic outside the protocol layer.

Adding a New Protocol

Multiplication examples

Testing Protocol

```
void multiply_a(x, y, &z) {
    z = x * y;
}
```

Semihonest Beaver (2PC)

```
void multiply_a(x, y, &z) {
    auto [a, b, c] = BTgen->getNext(x.size());

    auto A = open_shares_a(x + a);
    auto B = open_shares_a(y + b);

    z = y * A - a * B + c;
}
```

Adding a New Protocol

Multiplication examples

Semihonest Replicated (3PC)

```
void multiply_a(x, y, &z) {
    size_t size = x.size();

    Vector local(size);
    this->...->randomize(local);

    local += x(0) * y(0);
    local += x(0) * y(1);
    local += x(1) * y(0);

    Vector remote(size);
    exchange(local, remote);

    z(0) = local;
    z(1) = remote;
}
```

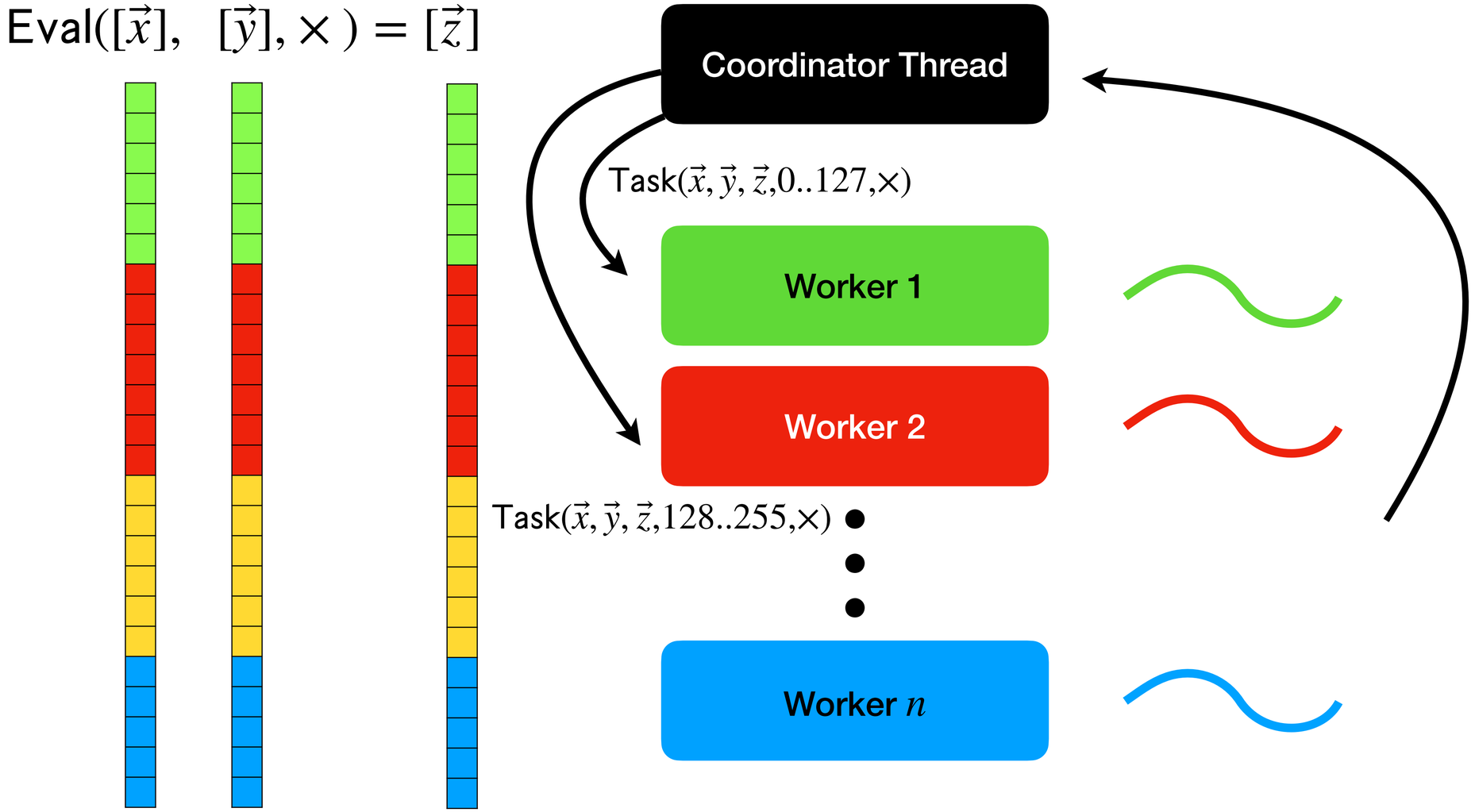
Adding a New Protocol

Multiplication examples

Malicious Replicated (4PC)

```
void multiply_a(x, y, &z) {
    int Pi, Pj, Pg, Ph;
    EVector r(x.size());

    for (pairs Pi, Pj) {
        Pg = next_party(Pi, Pj);
        Ph = excluded_party(Pi, Pj, Pg);

        if (this->partyID == Pg || this->partyID == Ph) {
            r += inp(r(0), Pi, Pj, Pg, Ph);
        } else {
            r += inp(x(Ph) * y(Pg) + x(Pg) * y(Ph), Pi, Pj, Pg, Ph);
        }
    }

    z = r + x * y;
}
```

Data-parallel Runtime



Data-parallel Runtime